The Fine Art of Commenting Cheat Sheet

The Cardinal Rule

> *"The best comment is the one you didn't have to write because your code was clear enough."*

When NOT to Comment

Don't Comment	Why	Better Alternative
What the code does	Code should be self-explanatory	Use descriptive names
Obvious operations	Adds noise	Delete the comment
Commented-out code	Creates confusion	Use version control
TODO without owner	Never gets done	Add name and date

```
// ✗ Bad: Increment counter by one
counter++;

// ✗ Bad: Check if user is null
if (user == null)

// ✓ Good: No comment needed - code is clear
var activeCustomers = customers.Where(c => c.IsActive);
```

When TO Comment

Do Comment	Example
Why (not what)	`// Using insertion sort because n < 10`
Workarounds	`// API returns null for empty, not empty array`
Warnings	`// WARNING: Not thread-safe`
Complex algorithms	`// Implements Dijkstra's shortest path`
Business rules	`// Tax exempt for orders over $1000 (Policy #123)`
External references	`// See RFC 7231 Section 6.5.1`

Comment Types

Single-Line Comments

```
// Use for brief explanations
int retryCount = 3; // Max retries before timeout
```

Multi-Line Comments

```
/*
 * Use sparingly for longer explanations.
 * Prefer breaking into smaller, self-documenting methods.
 */
```

XML Documentation (Public APIs)

```
/// <summary>
/// Calculates the total price including tax and discounts.
/// </summary>
/// <param name="items">The items to calculate.</param>
/// <param name="taxRate">Tax rate as decimal (e.g., 0.08 for 8%).</param>
/// <returns>Total price rounded to 2 decimal places.</returns>
/// <exception cref="ArgumentNullException">Thrown when items is null.
</exception>
/// <example>
/// <code>
/// var total = CalculateTotal(cartItems, 0.08m);
/// </code>
/// </example>
public decimal CalculateTotal(IEnumerable<Item> items, decimal taxRate)
```

XML Documentation Tags

Tag	Purpose
`<summary>`	Brief description
`<param>`	Parameter description
`<returns>`	Return value description
`<exception>`	Exceptions that may be thrown
`<example>`	Usage example
`<remarks>`	Additional details
`<see cref="">`	Cross-reference to other members
`<seealso>`	Related topics

Comment Maintenance

Keep Comments Updated

```
// ✗ Outdated comment (code changed, comment didn't)
// Returns the user's full name
public string GetEmail() { ... }

// ✓ Comment matches code
// Returns the user's email address
public string GetEmail() { ... }
```

Delete Dead Comments

```
// ✗ Don't do this
// int oldValue = 5;
// if (oldValue > 3) { ... }

// ✓ Just delete it - use git for history
```

Quick Checklist

Before committing, ask for each comment:

- [ ] Does this explain WHY, not WHAT?
- [ ] Would renaming make this comment unnecessary?
- [ ] Is this comment still accurate?
- [ ] Is this a TODO with owner and date?
- [ ] For public APIs: Is XML documentation complete?

The Smell Test

If you find yourself writing a comment, first ask:

1. Can I rename the variable/method to be clearer?
2. Can I extract a well-named method?
3. Can I simplify the logic?

Only if the answer is “no” to all three, write the comment.

Generated from Ask Marilyn About Software Testing - The Fine Art of Commenting Course